ПО ПРИЗНАКАМ
НОВАЯ ВЕРСИЯ

Живое объяснение 27 признаков EH/Unwind Analyzer: откуда они взялись, почему их ровно столько в версии 1.0.0, что каждый признак проверяет и как не сделать из него лишний вывод.

Этот файл написан для защиты и передачи проекта. Он не заменяет исходный код и матрицу evidence, а переводит их на нормальный человеческий язык.

Основа: `docs/evidence_matrix.md`, `src/heuristics.c`, `docs/ru/учебник по признакам/Список_27_признаков.csv`.

1. Откуда взялись 27 признаков

В проекте слово 'признак' означает не ярлык 'вредоносно', а наблюдаемый технический сигнал. Инструмент смотрит на Windows PE32+ AMD64 файл и задает очень практичный вопрос: можно ли доверять его таблицам исключений и unwind metadata, или в них есть странности, которые аналитику нужно открыть руками.

Число 27 не магическое. Это полный набор именованных findings, которые текущая версия 1.0.0 реально умеет выдавать в статической эвристической оценке. Их не выбирали ради красивого количества. Наоборот, похожие вещи объединяли, а отдельными оставляли только те сигналы, которые отличаются по смыслу, проверке или силе вывода.

• 13 признаков проверяют базовую целостность PE-секций и таблицы RUNTIME_FUNCTION. Это фундамент: если карта функций сломана, дальше анализировать опасно.

• 3 признака относятся к целостности UNWIND_INFO: читается ли оно, не выглядит ли странно, и не подтверждает ли оно проблемы размещения.

• 3 признака смотрят на массовое переиспользование unwind metadata. Такое бывает и законно, поэтому эти правила осторожные.

• 1 признак смотрит, не доминирует ли один exception handler слишком сильно.

• 7 признаков описывают размещение metadata и handler: где они лежат, какие права у секций и усиливают ли они друг друга.

Если коротко: сначала проект проверяет, не врут ли таблицы; потом - можно ли разобрать unwind metadata; потом - не лежит ли metadata в странном месте; потом - нет ли массовых повторов и централизованных handler. Последние признаки - комбинированные: они появляются, когда две отдельные странности подтверждают друг друга.

2. Как признаки отбирались

• Признак должен быть измеримым. Не 'выглядит подозрительно', а конкретно: диапазон вышел за SizeOfImage, handler попал в writable-секцию, таблица не отсортирована.

• Признак должен иметь связь с механизмом Windows x64 exception/unwind, PE-структурой или ручной проверкой в reverse engineering.

• Признак не должен раздувать ложные выводы. Поэтому есть ослабления: маленькая выборка, одинокий сигнал, только необычное имя секции без других проблем.

• Признак должен объясняться человеку. В отчете есть code, severity, family, affected count и representative RVA, чтобы аналитик мог открыть конкретное место.

• Похожие факты объединялись. Например, разные мелкие странности UNWIND_INFO сведены в unwind_anomalies, а не размножены в десятки мелких правил.

• Комбинации выделены отдельно только там, где вместе два факта реально говорят больше, чем по отдельности.

3. Как читать score и серьезность

Score в этом проекте - не процент вероятности malware. Это инженерная шкала 0..100: насколько много и насколько тяжелых сигналов набралось. Confidence - тоже не статистическая вероятность, а уверенность в интерпретации с учетом числа семейств, силы признаков и ослабляющих правил.

• Один средний признак - повод посмотреть, но не повод драматизировать.

• Высокий признак по целостности таблиц или handler обычно важнее, потому что ломает базовую модель файла.

• Несколько признаков из разных семейств говорят сильнее, чем несколько похожих признаков из одной группы.

• Маленькие файлы оцениваются осторожнее: проценты на малом количестве функций легко выглядят слишком громко.

• Комбо-признаки нужны для объяснения связки: проблема не случайно торчит в одном месте, а подтверждается рядом.

4. Быстрая карта всех признаков

N	Код признака	Семейство	Вес	Смысл простыми словами
1	`pe_section_raw_oob`	целостность PE-таблиц	+25	Секция в заголовках обещает байты, которых в файле уже нет.
2	`pe_section_virtual_oob`	целостность PE-таблиц	+20	Секция после загрузки как будто залезает за границы объявленного образа.
3	`pe_section_raw_overlap`	целостность PE-таблиц	+15	Две секции делят одни и те же байты на диске.
4	`pe_section_virtual_overlap`	целостность PE-таблиц	+20	Две секции претендуют на один и тот же участок памяти.
5	`pe_section_misaligned`	целостность PE-таблиц	+8	Размеры или адреса секций не согласованы с правилами выравнивания PE.
6	`pe_exception_directory_oob`	целостность PE-таблиц	+25	Каталог исключений, где должна лежать .pdata, указывает не туда или не читается.
7	`no_runtime_functions`	целостность PE-таблиц	+10	В x64-образе с кодом не нашлось записей RUNTIME_FUNCTION.
8	`invalid_ranges`	целостность PE-таблиц	+10 / +20	В записи функции конец стоит раньше начала или равен началу.
9	`rf_oob`	целостность PE-таблиц	+10 / +25	Функция из .pdata выходит за границы образа или понятного секционного контекста.
10	`unwind_rva_oob`	целостность PE-таблиц	+10 / +25	Запись функции указывает на UNWIND_INFO, который невозможно нормально найти в файле.
11	`rf_unsorted`	целостность PE-таблиц	+15	Таблица функций идет не по возрастанию BeginAddress.
12	`rf_overlaps`	целостность PE-таблиц	+25	Диапазоны функций пересекаются.
13	`duplicate_begin`	целостность PE-таблиц	+10	Несколько записей начинаются с одного и того же адреса функции.
14	`unwind_parse_fail`	целостность unwind-описаний	+10 / +25	UNWIND_INFO не удается разобрать до конца по правилам Windows x64.

N	Код признака	Семейство	Вес	Смысл простыми словами
15	`unwind_anomalies`	целостность unwind-описаний	+10 / +20	UNWIND_INFO читается, но внутри есть странные версии, операции, операнды, цепочки или ABI-паттерны.
16	`unwind_reuse_moderate`	массовое повторное использование unwind metadata	+10	Заметная доля функций делит один и тот же UNWIND_INFO, и рядом уже есть другой статический сигнал.
17	`unwind_reuse_strong`	массовое повторное использование unwind metadata	+20	Слишком много функций ссылается на один и тот же UNWIND_INFO.
18	`handler_dominance`	доминирование одного обработчика	+15	Один обработчик исключений обслуживает подозрительно большую часть функций.
19	`unwind_in_exec_section`	размещение metadata и handler	+20	UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код.
20	`unwind_unusual_section`	размещение metadata и handler	+8 / +15	UNWIND_INFO лежит не в привычной секции вроде .xdata, .rdata или .pdata.
21	`unwind_writable_section`	размещение metadata и handler	+10 / +18	UNWIND_INFO лежит в секции, которую можно изменять во время работы.
22	`handler_nonexec_section`	размещение metadata и handler	+15	Handler указывает туда, где код исполняться не должен.
23	`handler_writable_section`	размещение metadata и handler	+10	Handler указывает в секцию с правом записи.
24	`placement_exec_nonexec_handler_combo`	размещение metadata и handler	+10	Unwind metadata лежит как код, а handler указывает не в код.
25	`placement_integrity_combo`	целостность unwind-описаний	+8	Странное размещение metadata совпало с поломкой или странностью UNWIND_INFO.
26	`reuse_dominance_combo`	массовое повторное использование unwind metadata	+8	Массовое переиспользование unwind metadata совпало с доминированием одного handler.
27	`writable_metadata_handler_combo`	размещение metadata и handler	+10	И unwind metadata, и handler оказались связаны с writable-контекстом.

5. Подробно по каждому признаку

1. `pe_section_raw_oob`

Коротко: Секция в заголовках обещает байты, которых в файле уже нет.

Что делает: Инструмент берет таблицу секций PE и проверяет, не выходит ли область секции на диске за конец файла.

За что отвечает: Этот признак защищает весь дальнейший анализ: если секция указывает мимо файла, нельзя спокойно верить данным, которые будто бы лежат в этой секции.

Как оценивать: Это сильный сигнал. Сначала надо исключить простой вариант: файл мог быть обрезан при копировании или скачивании. Если файл целый, такой заголовок похож на намеренно испорченную или нестандартную разметку. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +25.

Осторожность: Не надо сразу говорить, что файл вредоносный. Правильнее сказать: структура файла противоречит его реальному размеру, поэтому выводы по секциям надо проверять вручную.

2. `pe_section_virtual_oob`

Коротко: Секция после загрузки как будто залезает за границы объявленного образа.

Что делает: Проверяется, укладывается ли виртуальный диапазон секции в SizeOfImage - общий размер образа, который ожидает загрузчик Windows.

За что отвечает: Признак отвечает за честность карты памяти: RVA-адреса должны попадать в понятные участки образа.

Как оценивать: Обычно это высокий риск для анализа. Если виртуальная карта сломана, один и тот же адрес может читаться неверно или вообще не иметь нормального места в образе. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +20.

Осторожность: Иногда учебные или специально сгенерированные файлы бывают кривыми. Для настоящего вывода нужно открыть PE-заголовки и проверить SizeOfImage, SectionAlignment и границы секций.

3. `pe_section_raw_overlap`

Коротко: Две секции делят одни и те же байты на диске.

Что делает: Инструмент сравнивает raw-диапазоны секций и ищет пересечения.

За что отвечает: Это ловит ситуацию, где один кусок файла можно объяснить сразу двумя секциями. Для обычной сборки это странно.

Как оценивать: Средний сигнал. Сам по себе он не приговор, но он повышает внимание: упаковщики, защитные оболочки и ручные правки иногда используют такую неоднозначность. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +15.

Осторожность: Если это единственный признак и файл маленький, не надо разгонять вывод. Нужно посмотреть, какие именно секции пересекаются и используются ли они реально.

4. pe_section_virtual_overlap

Коротко: Две секции претендуют на один и тот же участок памяти.

Что делает: Проверяется пересечение виртуальных адресов секций после загрузки.

За что отвечает: Признак отвечает за однозначность адресного пространства PE: один RVA должен вести в один понятный участок.

Как оценивать: Это сильнее, чем raw-overlap. Если память размечена неоднозначно, анализ handler и unwind metadata может уехать в неправильное место. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +20.

Осторожность: Нужно сверить, как такой файл реально принимает загрузчик Windows. Но для статического анализа это уже красная зона.

5. pe_section_misaligned

Коротко: Размеры или адреса секций не согласованы с правилами выравнивания PE.

Что делает: Инструмент проверяет FileAlignment, SectionAlignment и связанные инварианты заголовков.

За что отвечает: Выравнивание - это бытовая дисциплина формата. Когда она нарушена, парсер уже не может читать образ как обычный compiler output.

Как оценивать: Средний сигнал. Он полезен как фон: один misalignment может быть особенностью генератора, но вместе с другими признаками становится важным подтверждением. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +8.

Осторожность: Сначала смотрят масштаб: одно небольшое отклонение и пачка сломанных секций - это разные истории.

6. pe_exception_directory_oob

Коротко: Каталог исключений, где должна лежать .pdata, указывает не туда или не читается.

Что делает: Проверяется Exception Directory из PE Data Directory: его RVA и размер должны переводиться в реальные байты.

За что отвечает: Это входная дверь всего EH/unwind анализа. Если она сломана, таблицу функций нельзя считать надежной.

Как оценивать: Высокий сигнал. Для Windows x64 корректная exception directory важна для нормального unwind поведения. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +25.

Осторожность: Если каталог отсутствует у файла без кода, это может быть нормально. Но если есть исполняемые секции, а exception directory сломана, это надо показывать как серьезный факт.

7. no_runtime_functions

Коротко: В x64-образе с кодом не нашлось записей RUNTIME_FUNCTION.

Что делает: Инструмент видит, что exception directory пустая, а исполняемый код в образе есть.

За что отвечает: Записи RUNTIME_FUNCTION помогают Windows понимать границы функций и раскручивать стек при исключениях.

Как оценивать: Это средний и осторожный признак. Маленький leaf-only код теоретически может обходиться без таких записей. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +10.

Осторожность: Нельзя оценивать его грубо. Надо посмотреть, есть ли сложные функции, вызовы, прологи, stack frame. Если код реально нетривиальный, отсутствие .pdata выглядит подозрительнее.

8. `invalid_ranges`

Коротко: В записи функции конец стоит раньше начала или равен началу.

Что делает: Для каждой RUNTIME_FUNCTION проверяется диапазон [BeginAddress, EndAddress).

За что отвечает: Признак отвечает за базовую геометрию функции: функция должна занимать ненулевой и направленный диапазон адресов.

Как оценивать: Если таких записей мало, это средний сигнал. Если больше 1 процента таблицы - уже высокий: это не похоже на случайную мелочь. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +20.

Осторожность: Нужно смотреть affected count и процент. Одна битая запись в огромном файле и системная поломка таблицы оцениваются по-разному.

9. `rf_oob`

Коротко: Функция из .pdata выходит за границы образа или понятного секционного контекста.

Что делает: BeginAddress и EndAddress сверяются с SizeOfImage и секциями.

За что отвечает: Этот признак ловит RUNTIME_FUNCTION, которая описывает код там, где кода в нормальной карте образа нет.

Как оценивать: До 1 процента - средний сигнал, выше - высокий. Чем больше таких записей, тем меньше доверия к таблице. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +25.

Осторожность: Важно проверить, не анализируется ли намеренно поврежденный тестовый образ. В реальном ПО такие диапазоны требуют ручной проверки.

10. `unwind_rva_oob`

Коротко: Запись функции указывает на UNWIND_INFO, который невозможно нормально найти в файле.

Что делает: UnwindInfoRVA переводится в file offset и секцию; если адрес не мапится, признак срабатывает.

За что отвечает: RUNTIME_FUNCTION без читаемого UNWIND_INFO теряет смысл: есть функция, но нет нормальной инструкции, как раскручивать стек.

Как оценивать: Порог такой же: мало - средний сигнал, системно - высокий. Особенно важно, если вместе есть parse fail или странное размещение metadata. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +25.

Осторожность: Не путать с отсутствием handler. Handler может не быть, а UNWIND_INFO все равно должен читаться.

11. `rf_unsorted`

Коротко: Таблица функций идет не по возрастанию BeginAddress.

Что делает: Инструмент проходит RUNTIME_FUNCTION и считает инверсии порядка.

За что отвечает: Windows и инструменты ожидают предсказуемую таблицу. Несортированность мешает быстрому и однозначному поиску функции по адресу.

Как оценивать: Средний сигнал. Он показывает не столько поломку одной функции, сколько грязную или искусственную организацию таблицы. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +15.

Осторожность: Если есть только этот признак, лучше говорить осторожно: таблица нестандартно упорядочена, нужна сверка с остальными симптомами.

12. `rf_overlaps`

Коротко: Диапазоны функций пересекаются.

Что делает: После сортировки инструмент смотрит, не залезает ли одна RUNTIME_FUNCTION в адреса другой.

За что отвечает: Один адрес кода не должен одновременно принадлежать двум нормальным функциям в этой таблице.

Как оценивать: Высокий сигнал, потому что пересечение ломает саму идею надежной карты функций. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +25.

Осторожность: Стоит открыть конкретные RVA и посмотреть, это ручная правка, защита, генератор кода или повреждение файла.

13. `duplicate_begin`

Коротко: Несколько записей начинаются с одного и того же адреса функции.

Что делает: Инструмент ищет одинаковые BeginAddress в RUNTIME_FUNCTION.

За что отвечает: Признак ловит дубли и неоднозначность: один старт функции получает несколько разных описаний.

Как оценивать: Средний сигнал. Он становится сильнее, если дубли ведут к разным unwind info или идут вместе с overlap. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +10.

Осторожность: Одинаковое начало не всегда объясняет всю картину. Нужно смотреть, чем отличаются записи и куда они указывают.

14. `unwind_parse_fail`

Коротко: UNWIND_INFO не удается разобрать до конца по правилам Windows x64.

Что делает: Парсер читает version, flags, opcodes, handler/chained tail и останавливается, если структура физически не сходится.

За что отвечает: Это один из главных признаков целостности: metadata есть, но ее нельзя прочитать как нормальное unwind-описание.

Как оценивать: До 1 процента - средний сигнал, больше - высокий. Массовые parse fail обычно говорят не о случайности, а о системной проблеме. В матрице проекта: семейство - целостность unwind-описаний, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +25.

Осторожность: Формальные ABI-предупреждения в strict mode не равны parse fail. Здесь речь именно о том, что структура не читается корректно.

15. `unwind_anomalies`

Коротко: UNWIND_INFO читается, но внутри есть странные версии, операции, операнды, цепочки или ABI-паттерны.

Что делает: Инструмент не падает на записи, а собирает необычные свойства: opcode, opinfo, chain, epilog и другие детали.

За что отвечает: Этот признак нужен для серой зоны: файл не сломан полностью, но его unwind metadata выглядит нетипично.

Как оценивать: Если затронуто до 5 процентов функций - средний сигнал. Если больше - высокий, потому что странность уже стала массовым шаблоном. В матрице проекта: семейство - целостность unwind-описаний, правило серьезности - medium` if `<=5%`, else `high, вклад в score - +10 / +20.

Осторожность: Нужно различать 'нестандартно' и 'невозможно'. Некоторые компиляторы дают необычные, но допустимые варианты.

16. `unwind_reuse_moderate`

Коротко: Заметная доля функций делит один и тот же UNWIND_INFO, и рядом уже есть другой статический сигнал.

Что делает: Инструмент считает, сколько функций ссылаются на каждый UnwindInfoRVA; умеренный reuse включается только при дополнительном подтверждении.

За что отвечает: Признак отвечает за поиск централизованной metadata-схемы, где много функций ведут в одну точку описания.

Как оценивать: Это средний сигнал и специально осторожный: он не срабатывает просто от reuse, если больше ничего странного нет. В матрице проекта: семейство - массовое повторное использование unwind metadata, правило серьезности - medium, вклад в score - +10.

Осторожность: Нормальные компиляторы тоже могут переиспользовать одинаковые unwind-описания. Поэтому здесь обязательно нужен соседний признак.

17. unwind_reuse_strong

Коротко: Слишком много функций ссылается на один и тот же UNWIND_INFO.

Что делает: Если в большом образе больше 30 процентов функций делят один UnwindInfoRVA, инструмент выдает сильный reuse-признак.

За что отвечает: Это ловит грубую централизацию unwind metadata, характерную для оберток, диспетчеров или искусственно построенных таблиц.

Как оценивать: Высокий сигнал, но только на достаточно большой таблице. В маленьком файле проценты легко обманывают. В матрице проекта: семейство - массовое повторное использование unwind metadata, правило серьезности - high, вклад в score - +20.

Осторожность: Нужно смотреть размер корпуса функций. На пяти функциях '40 процентов' может означать всего две записи.

18. handler_dominance

Коротко: Один обработчик исключений обслуживает подозрительно большую часть функций.

Что делает: Инструмент считает HandlerRVA и смотрит, не доминирует ли один handler среди большого числа записей.

За что отвечает: Обычная программа может иметь общие runtime handlers, но тотальное доминирование похоже на центральный диспетчер.

Как оценивать: Средний сигнал. В реализации он смягчен: если handler выглядит как нормальный runtime-код и нет других статических проблем, признак не раздувается. В матрице проекта: семейство - доминирование одного обработчика, правило серьезности - medium, вклад в score - +15.

Осторожность: На защите важно сказать честно: это не доказательство защиты или упаковщика, а повод проверить, почему один handler так часто повторяется.

19. unwind_in_exec_section

Коротко: UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код.

Что делает: Адрес unwind metadata связывается с секцией; если у секции стоит право execute, признак срабатывает.

За что отвечает: Metadata обычно живет в .xdata, .rdata, .pdata или похожих data-секциях. Исполняемая секция для нее выглядит странно.

Как оценивать: Высокий сигнал. Такое размещение труднее объяснить обычной компиляцией. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +20.

Осторожность: Нужно посмотреть имя секции и права. Если это специально названная, но executable-секция, право execute все равно остается важным фактом.

20. unwind_unusual_section

Коротко: UNWIND_INFO лежит не в привычной секции вроде .xdata, .rdata или .pdata.

Что делает: Инструмент проверяет имя и контекст секции, где находится unwind metadata.

За что отвечает: Признак отвечает за бытовую нормальность размещения: metadata должна лежать там, где ее ожидают инструменты и загрузчик.

Как оценивать: До 5 процентов - средний сигнал, больше - высокий. Массовое необычное размещение говорит сильнее, чем одна запись. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - medium` if `<=5%`, else `high, вклад в score - +8 / +15.

Осторожность: Этот признак специально может ослабляться, если он один. Странное имя секции еще не равно повреждению.

21. unwind_writable_section

Коротко: UNWIND_INFO лежит в секции, которую можно изменять во время работы.

Что делает: Проверяется флаг write у секции, где лежит unwind metadata.

За что отвечает: Unwind metadata обычно воспринимается как описание, а не как изменяемые рабочие данные. Writable-контекст повышает риск подмены.

Как оценивать: Средний или высокий сигнал: больше 5 процентов затронутых записей - уже широкий шаблон. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - medium` if `<=5%`, else `high, вклад в score - +10 / +18.

Осторожность: Некоторые runtime-сценарии могут строить таблицы динамически, но для статического файла writable metadata - веская причина открыть объект руками.

22. handler_nonexec_section

Коротко: Handler указывает туда, где код исполняться не должен.

Что делает: HandlerRVA связывается с секцией; если секция не executable или вообще не находится, признак срабатывает.

За что отвечает: Обработчик исключения - это код. Если адрес handler ведет в неисполняемые данные, что-то в модели файла не сходится.

Как оценивать: Высокий сигнал. Это один из самых понятных для объяснения признаков размещения. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +15.

Осторожность: Стоит проверить, не ошибка ли это парсинга из-за уже сломанной UNWIND_INFO. Если parse fail рядом есть, признаки усиливают друг друга.

23. handler_writable_section

Коротко: Handler указывает в секцию с правом записи.

Что делает: Инструмент проверяет write-флаг у секции, куда попадает HandlerRVA.

За что отвечает: Код handler в writable-секции выглядит менее надежно: теоретически его можно менять как данные.

Как оценивать: Средний сигнал. Сам по себе он не такой сильный, как non-exec handler, но вместе с writable unwind metadata становится важнее. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - medium, вклад в score - +10.

Осторожность: Нужно смотреть сочетания прав секции: execute+write и просто write без execute - разные случаи.

24. `placement_exec_nonexec_handler_combo`

Коротко: Unwind metadata лежит как код, а handler указывает не в код.

Что делает: Это комбинированный признак: он появляется, когда одновременно есть unwind_in_exec_section и handler_nonexec_section.

За что отвечает: Комбинация показывает, что странность не одиночная: обе стороны exception-механизма размещены нелогично.

Как оценивать: Высокий сигнал. Его стоит объяснять как усиление: два связанных факта вместе говорят больше, чем каждый по отдельности. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +10.

Осторожность: Комбо не заменяет ручной анализ. Оно говорит: 'смотри на пару unwind metadata + handler как на один подозрительный узел'.

25. `placement_integrity_combo`

Коротко: Странное размещение metadata совпало с поломкой или странностью UNWIND_INFO.

Что делает: Признак появляется, когда placement-аномалия идет вместе с parse fail, unwind anomalies или недоступным UnwindInfoRVA.

За что отвечает: Он отделяет просто необычное место от ситуации, где необычное место еще и плохо читается.

Как оценивать: Средний усилитель. Он не самый тяжелый по весу, но хорошо повышает доверие к тому, что проблема настоящая. В матрице проекта: семейство - целостность unwind-описаний, правило серьезности - medium, вклад в score - +8.

Осторожность: Важно называть его именно подтверждающим признаком. Он не первичный факт, а связка двух фактов.

26. `reuse_dominance_combo`

Коротко: Массовое переиспользование unwind metadata совпало с доминированием одного handler.

Что делает: Инструмент связывает две статистики: много функций делят один UnwindInfoRVA и много функций идут к одному HandlerRVA.

За что отвечает: Такой рисунок похож на центральный диспетчер: много функций формально разные, но exception-механизм у них сведен в одну схему.

Как оценивать: Средний усилитель. Он ценен не весом сам по себе, а тем, что соединяет reuse и handler behavior. В матрице проекта: семейство - массовое повторное использование unwind metadata, правило серьезности - medium, вклад в score - +8.

Осторожность: Нужно проверять на крупных файлах. На маленьких выборках доли могут выглядеть громче, чем есть на самом деле.

27. `writable_metadata_handler_combo`

Коротко: И unwind metadata, и handler оказались связаны с writable-контекстом.

Что делает: Комбо срабатывает, когда одновременно есть writable unwind metadata и writable handler target.

За что отвечает: Обе части механизма исключений оказываются изменяемыми. Для обычного статического образа это трудно назвать спокойной картиной.

Как оценивать: Высокий сигнал. Его удобно объяснять как связанное подтверждение: меняемой выглядит не одна случайная деталь, а вся линия metadata -> handler. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +10.

Осторожность: Ручная проверка обязательна: нужно посмотреть права секций, конкретные RVA и то, является ли файл обычным PE или специальным runtime-генератором.

6. Что говорить на защите

Самая честная формулировка такая: проект не пытается 'угадать вредоносность'. Он строит проверяемую карту EH/unwind metadata и отмечает места, где эта карта противоречит нормальной структуре Windows PE32+ AMD64 или выглядит нетипично для обычной компиляции.

27 признаков - это не случайная цифра и не маркетинговый список. Это набор устойчивых, реализованных и документированных сигналов версии 1.0.0. Они покрывают пять смысловых зон: целостность таблиц, целостность UNWIND_INFO, размещение metadata, повторное использование metadata и доминирование обработчика.

Правильный вывод по отчету звучит не 'score 70 значит 70 процентов опасности', а 'в файле набралось много технических сигналов, часть из них сильные, они затрагивают такие-то семейства, поэтому требуется ручная проверка конкретных RVA и секций'.

Именно поэтому в проекте рядом со score есть severity, confidence, affected count, representative RVA, mitigations и manual validation steps. Машина не заменяет аналитика, а быстро подсвечивает места, куда аналитику стоит смотреть первым.

Приложение: метрики проекта и как их защищать

Короткий ответ на защите: в проекте есть конкретные метрики, но они не являются статистической вероятностью вредоносности. Это инженерные triage-метрики для приоритизации ручного анализа: счетчики структурных нарушений, доли, score 0..100, suspicion = score / 100, confidence, evidence level, dynamic signal count, case strength и флаг requires_manual_review. Окончательный вывод делает реверс-инженер по цепочке evidence: PE sections -> RUNTIME_FUNCTION -> UNWIND_INFO -> finding -> report -> ручная проверка.

1. Какие метрики реально есть

Группа	Метрики	Где видны	Что означают
Структурные счетчики	section_raw_oob, section_virtual_oob, overlaps, misaligned, rf_invalid_ranges, rf_oob, parse_fail	JSON: pe_validation, heuristic_analysis	Сколько конкретных структурных проблем найдено в PE/.pdata/UNWIND_INFO.
Доли и ratios	unwind_reuse_ratio, unwind_unique_ratio, handler_reuse_ratio; проценты parse/range проблем	JSON и finding messages	Насколько проблема массовая, а не единичная.
Score	score 0..100	heuristic_analysis.score, HTML card, console	Сумма весов сработавших признаков, ограниченная 0..100.
Suspicion	suspicion = score / 100	heuristic_analysis.suspicion	Нормированная форма score. Это не вероятность.
Confidence	confidence 0.05..0.90 static, final_confidence до 0.98	heuristic_analysis.confidence, assessment.final_confidence	Уверенность в интерпретации evidence с учетом разнообразия сигналов и ослабляющих правил.
Evidence level	none, weak, medium, strong	heuristic_analysis.evidence_level, assessment	Качественная сила evidence по счетчикам severity и score.
Dynamic metrics	exceptions_seen, handler hits, VEH/VCH/UEF, RtlAddFunctionTable, dynamic_signal_count	dynamic_validation, assessment	Что реально наблюдалось при запуске target под Debug API.
Итоговая классификация	verdict, final_verdict, case_strength, static_dynamic_correlation	heuristic_analysis, assessment	Техническая сводка для triage, не автоматический приговор.

Главный вывод

- Метрики есть и они формализованы в коде и JSON schema.
- Метрики являются эвристическими, инженерными, объяснимыми и воспроизводимыми.
- Проект не утверждает ROC/AUC-калибровку, вероятность malware или универсальные пороги для всех packer/protector families.
- Реверс-инженер обязан читать findings, representative RVA, section context, weakening rules и manual_validation_steps.

2. Как считается score 0..100

Score начинается с 0. Каждый сработавший finding добавляет фиксированный вес. После добавления весов значение ограничивается диапазоном 0..100. Поэтому score - это шкала подозрительности по инженерной матрице, а не статистическая вероятность.

Finding / ситуация	Вес	Комментарий
pe_section_raw_oob	+25	Секция указывает raw range за пределами файла.
pe_section_virtual_oob	+20	Virtual range секции выходит за SizeOfImage.
pe_section_raw_overlap	+15	Raw ranges секций пересекаются.
pe_section_virtual_overlap	+20	Virtual ranges секций пересекаются.
pe_section_misaligned	+8	Нарушены alignment/header invariants.
pe_exception_directory_oob	+25	Exception directory отсутствует, неполон или нечитабелен при заявленном наличии.
no_runtime_functions	+10	Нет .pdata при наличии executable sections; может быть benign leaf-only.
invalid_ranges	+10 / +20	End <= Begin; high если доля > 1%.
rf_oob	+10 / +25	Begin/End вне SizeOfImage; high если доля > 1%.
unwind_rva_oob	+10 / +25	UnwindInfoRVA отсутствует или вне image; high если доля > 1%.
rf_unsorted	+15	Таблица не отсортирована по BeginAddress.
rf_overlaps	+25	Диапазоны функций перекрываются после сортировки.
duplicate_begin	+10	Дублируется BeginAddress.
unwind_parse_fail	+10 / +25	UNWIND_INFO не парсится; high если доля > 1%.
unwind_anomalies	+10 / +20	Аномалии в version/opcode/chain/etc.; high если доля > 5%.
unwind_reuse_moderate	+10	Reuse > 10%, n > 200 и есть подтверждающий статический сигнал.
unwind_reuse_strong	+20	Reuse > 30% при n > 200.
handler_dominance	+15	Один handler доминирует: handler_count > 50 и reuse ratio > 0.80.
unwind_in_exec_section	+20	Unwind metadata лежит в executable section.
unwind_unusual_section	+8 / +15	Unwind metadata не в xdata/rdata/pdata-like section; high если > 5%.
unwind_writable_section	+10 / +18	Unwind metadata лежит в writable section; high если > 5%.
handler_nonexec_section	+15	Handler target в non-exec или неизвестной section.
handler_writable_section	+10	Handler target в writable section.
placement_exec_nonexec_handler_combo	+10	Комбинация executable unwind metadata и non-exec handler.
placement_integrity_combo	+8	Placement anomaly подтверждена integrity-проблемой.
reuse_dominance_combo	+8	Heavy unwind reuse совпал с dominant handler.
writable_metadata_handler_combo	+10	Writable unwind metadata совпала с writable handler target.

Пороги verdict

Score	verdict	Как читать
0..14	no_strong_static_anomalies	Сильных статических признаков нет, но это не доказывает benign.
15..39	low_suspicion	Есть слабое/локальное основание для проверки.

Score	verdict	Как читать
40..69	moderate_suspicion	Есть значимый набор признаков, нужен разбор в Ghidra/PE viewer.
70..100	high_suspicion	Сильная структурная картина, ручная проверка обязательна.

3. Как считать доли и ratios

Многие признаки важны не только фактом срабатывания, но и масштабом. Одна странная запись в маленьком файле и массовая поломка таблицы - разные ситуации. Поэтому проект считает счетчики и отношения.

Метрика	Формула	Интерпретация
invalid_ranges_pct	rf_invalid_ranges / total	Доля записей .pdata, где EndAddress <= BeginAddress.
oob_pct	rf_oob / total	Доля функций, чей begin/end выходит за SizeOfImage.
unwind_oob_pct	rf_unwind_oob / total	Доля записей с unmappable/out-of-image UnwindInfoRVA.
unwind_fail_pct	unwind_parse_fail / total	Доля UNWIND_INFO, которые не удалось разобрать.
unwind_anom_pct	unwind_anomalies / total	Доля записей с нефомальными unwind anomalies.
unwind_reuse_ratio	max_count_same_UnwindInfoRVA / total	Насколько много функций ссылается на один и тот же unwind blob.
unwind_unique_ratio	unique_UnwindInfoRVA / total	Насколько разнообразна unwind metadata.
handler_reuse_ratio	max_count_same_HandlerRVA / handler_count	Насколько один handler доминирует среди функций с handler.

Пороговые правила

- 1% для invalid_ranges, rf_oob, unwind_rva_oob и unwind_parse_fail: выше этого проблема считается не единичной.
- 5% для unwind_anomalies и placement prevalence: выше этого видна широкая структурная картина.
- 10% для moderate unwind reuse: учитывается только при total > 200 и наличии другого статического подтверждения.
- 30% для strong unwind reuse: много функций указывают на один UnwindInfoRVA при total > 200.
- 80% для handler dominance: один handler покрывает подавляющее большинство функций с handler, но benign runtime-like context может подавить finding.

Как объяснить на защите

Если спросили, почему используются проценты: потому что абсолютный счетчик без denominator вводит в заблуждение. Например, 1 parse failure из 3 функций и 1 parse failure из 300 функций имеют разный аналитический вес. В отчете поэтому надо смотреть одновременно total, affected_count и ratio.

4. Confidence: уверенность, а не вероятность

Confidence показывает не вероятность вредоносности, а уверенность проекта в интерпретации статического verdict. Он растет, когда evidence разнообразно и сильно, и уменьшается, когда сигнал одиночный, выборка маленькая или есть benign explanation.

Компонент	Значение
База	0.15
Если score == 0	0.20 перед penalty
За каждое evidence family	+0.11
За каждый high/critical finding	+0.08
За каждый medium finding	+0.04
Hard table-integrity bonus	+0.08 при invalid ranges / rf_oob / unwind_oob / overlaps
Hard unwind-integrity bonus	+0.08 при parse_fail / unwind_anomalies
single_signal_only	-0.12 если один non-strong finding
small_sample_size	-0.06 если total < 32, findings есть, strong нет
placement_only_signal	-0.10 если есть только unusual placement без integrity/exec/writable context
single_family_only	-0.08 если evidence из одной family и strong нет
Static clamp	0.05..0.90

Final confidence

В блоке assessment final_confidence стартует от static confidence. Если dynamic session завершена полноценно и есть dynamic signals, добавляется +0.10. Если есть static/dynamic correlation, добавляется еще +0.10. Верхняя граница final_confidence - 0.98. Это все еще не probability; это удобная шкала надежности evidence chain.

5. Evidence level, case strength и dynamic metrics

Static evidence level

Level	Правило
strong	strong_count >= 2 или score >= 70
medium	strong_count >= 1 или medium_count >= 2 или score >= 30
weak	есть findings или score > 0, но нет medium/strong условий
none	finding_count == 0 и score == 0

Dynamic signal count

Категория	Когда добавляет 1 к dynamic_signal_count
unwind handler	unwind_handler_hits > 0
VEH	veh_hits > 0 или veh_registrations > 0
VCH	vch_hits > 0 или vch_registrations > 0
UEF	uef_hits > 0 или uef_set_calls > 0
Dynamic function tables	rtl_add_function_table_calls > 0 или rtl_install_function_table_cb_calls > 0
Second chance	second_chance_seen > 0

Runtime evidence level

- strong: handler hit и минимум один дополнительный medium runtime signal.
- medium: handler hit или минимум два medium runtime signals.
- weak: только слабая runtime activity, например first-chance exceptions или registrations.
- none: runtime activity нет или dynamic mode не запускался.
- Если session неполная, timeout или instrumentation incomplete, runtime evidence cap = weak.

Case strength

case_strength	Правило
strong_correlated_case	Есть static/dynamic correlation и static или runtime evidence = strong.
medium_correlated_case	Есть correlation и static или runtime evidence = medium.
strong_unconfirmed_case	Strong static/runtime evidence без cross-layer correlation.
medium_case	Medium static/runtime evidence.
weak_case	Только weak evidence.
no_case	Нет meaningful evidence.

6. Метрики качества проекта и валидации

Кроме метрик одного анализа, в проекте есть validation/evidence метрики качества: сколько тестов прошло, сколько отчетов валидны, сколько файлов corpus flagged, время анализа и outcomes dynamic corpus. Они нужны для защиты работоспособности инструмента, но не заменяют per-file score.

Метрика качества	Пример из evidence 1.0.0	Смысл
CTest	MinGW Release 16/16 PASS, MSVC Release 16/16 PASS	Сборка и unit/regression проверки проходят.
Real benign static corpus	106/106 PASS, score 0	Совместимость на реальных benign PE без срабатываний.
Labeled static corpus	14/14 PASS, 12 anomalous flagged	Синтетические признаки обнаруживаются.
Dynamic smoke	4/4 PASS	Контролируемые VEH/VCH/UEF/RtlFunctionTable сценарии работают.
Dynamic full corpus	124/124 valid	Динамические отчеты структурно валидны.
Performance	median 9 ms, p95 10 ms, max 15 ms на labeled static corpus	Static анализ быстрый на тестовом наборе.

Чего пока нет

- Нет заявленной ROC/AUC калибровки.
- Нет утверждения, что score = вероятность malware.
- Нет универсального threshold для всех компиляторов, protectors и JIT/runtime scenarios.
- Есть scaffold `tests/score_sweep.py`, который позволяет расширять corpus и сравнивать пороги в будущей работе.

7. Как читать отчет по метрикам

1. Сначала проверь report_metadata: версия tool/schema, SHA-256 input, command line, strict_abi/follow_chain.
2. Посмотри total runtime functions. Маленький total усиливает риск переинтерпретации ratios.
3. Открой pe_validation: если уже PE layout сломан, дальнейшие признаки читаются в этом контексте.
4. Прочитай heuristic_analysis.score, suspicion, verdict, evidence_level.
5. Не останавливайся на score. Открой findings: code, family, severity, affected_count, representative_rva, representative_section.
6. Сравни counters и ratios: массовая ли проблема или один локальный случай.
7. Проверь confidence и weakening_rules_applied. Если есть mitigation, вывод должен звучать осторожнее.
8. Если dynamic_validation есть, сначала проверь ok/timed_out/target_exited/instrumentation_complete.
9. Потом смотри dynamic_signal_count, runtime_evidence_level и static_dynamic_correlation.
10. Заверши manual_validation_steps: Ghidra, PE viewer, hex view, debugger/lab.

Примеры формулировок

- Score 0: инструмент не нашел сильных статических anomalies; это не доказательство benign semantics.
- Low score + low confidence: есть одиночный сигнал, его надо проверить вручную, особенно если sample small.
- High score + strong evidence: структура PE/unwind сильно отличается от нормальной, но причина устанавливается manual validation.
- Runtime signals only: динамика что-то показала, но статической корреляции нет; это отдельное наблюдение, не полный proof graph.
- Strong correlated case: лучший сценарий evidence, потому что static suspicious context подтвержден runtime behavior.

8. Ответы на вопросы комиссии

Вопрос	Короткий ответ
Есть ли в проекте метрики?	Да. Есть счетчики, ratios, score 0..100, suspicion, confidence, evidence_level, dynamic_signal_count, case_strength и validation metrics.
Это вероятность вредоносности?	Нет. Score и confidence являются инженерными эвристическими шкалами. Проект не заявляет статистическую вероятность.
Кто принимает итоговое решение?	Реверс-инженер. Проект дает structured evidence и manual validation plan, а не автоматический юридический/вредоносный verdict.
Как считается score?	Аддитивно по матрице весов findings с clamp 0..100. Например parse fail, OOB, suspicious placement, reuse/dominance и combo-сигналы добавляют разные веса.
Как оценивать confidence?	По разнообразию и силе evidence: families, strong/medium findings, hard integrity bonuses; затем применяются weakening rules.
Почему нужны weakening rules?	Чтобы одиночный сигнал, маленькая выборка или benign placement не выглядели слишком уверенно.
Что такое case_strength?	Итоговый класс evidence: no/weak/medium/strong_unconfirmed/medium_correlated/strong_correlated case.
Как доказать воспроизводимость?	Показать JSON schema, report metadata, SHA-256 input, CTest gates, corpus summaries и score_sweep scaffold.

Финальная защитная формулировка

В работе используются не ML-метрики классификации, а объяснимые инженерные метрики анализа EH/Unwind metadata. Они позволяют количественно ранжировать подозрительность, показать силу и разнообразие evidence, отделить статические признаки от runtime observations и сформировать план ручной валидации. Поэтому корректный вывод звучит так: инструмент не заменяет реверс-инженера, а дает воспроизводимую evidence-based основу для его решения.